SPI Master with Wishbone Interface

Design description, RTL debug and verification report

Syed Zayed · October 2026 ·
github.com/SyedZayedswsjz/wishbone-spi-master-rtl2gds

Origin and credit. The RTL is based on the OpenCores SPI Master core by Simon Srot (LGPL-2.1). The top-level block diagram (Figure 1) and the reference timing diagram (Figure 6) follow the OpenCores SPI core specification. **The internal architecture diagrams (Figures 2–5), the bug fixes, the self-checking testbench and simulation results (Sections 8–9), the SDC timing constraints and the Cadence implementation scripts are my own work.**

1. Introduction

In System-on-Chip (SoC) designs, processors must communicate efficiently with peripheral devices. Standard on-chip buses and peripheral interfaces make IP blocks reusable and easy to integrate. Two widely used standards are the **Wishbone bus** and the **Serial Peripheral Interface (SPI)**. Combining them gives an **SPI–Wishbone interface**: an SPI controller that a processor can program over the Wishbone bus.

Wishbone is an open-source, synchronous on-chip bus from OpenCores. It connects a **bus master** (such as a CPU or DMA controller) to one or more **bus slaves** (memories or peripheral controllers) using simple handshaking signals: cycle, strobe, acknowledge and error. It has no licensing restrictions, which makes it popular in academic and open-source hardware.

SPI is a synchronous, full-duplex serial protocol for short-distance, high-speed links between a master and one or more slaves. It uses four signals: Serial Clock (SCLK), Master Out Slave In (MOSI), Master In Slave Out (MISO) and Slave Select (SS). It is commonly used for sensors, ADCs, DACs, EEPROMs and displays.

In this design, the SPI controller is a **Wishbone slave IP core**: it bridges the on-chip Wishbone bus and off-chip SPI devices.

2. Top-Level Architecture

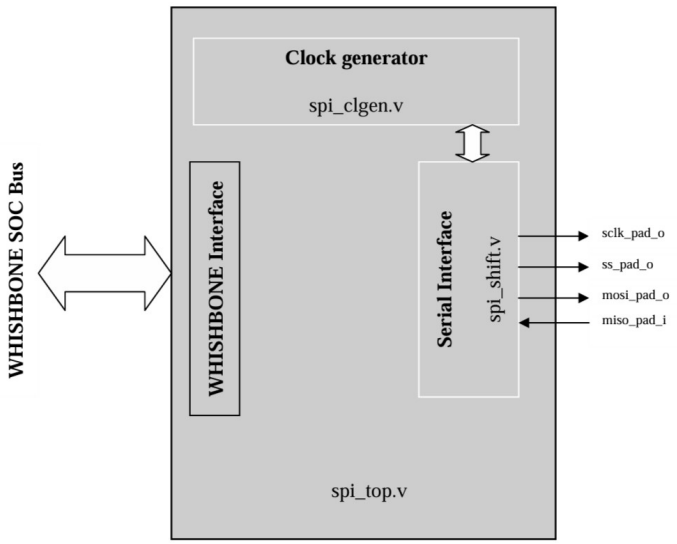


Figure 1. SPI Master core block diagram (adapted from the OpenCores SPI specification).

The top-level module, `spi_top.v`, connects three blocks: the Wishbone interface (register file and bus handshake), the clock generator (`spi_clgen.v`) and the serial interface (`spi_shift.v`). Separating bus control, clock generation and serial data handling keeps each block simple and easy to verify.

The **Wishbone interface** decodes the bus address and maps reads and writes onto the internal SPI registers. The **clock generator** divides the Wishbone clock by a programmable value to produce SCLK, and emits one-cycle pulses marking each SCLK rising and falling edge. The **serial interface** uses those pulses to shift data out on MOSI and sample data in from MISO. The clock generator runs only while the shift register reports a transfer in progress (`tip`).

> **Design note: a single clock domain.** Every flip-flop is clocked by `wb_clk_i`. SCLK is produced as a registered output and is never used as a clock inside the core; the shift logic uses the `pos_edge` and `neg_edge` enable pulses instead. This avoids clock-domain crossings and keeps static timing analysis simple.

3. Wishbone Interface

3.1 Signals

Port	Width	Direction	Description
`wb_clk_i`	1	Input	Master clock. All logic in the core runs on this clock.
`wb_rst_i`	1	Input	**Asynchronous** reset, active high.
`wb_adr_i`	5	Input	Register select (word address, see Section 3.2).
`wb_dat_i`	32	Input	Write data from the bus master.
`wb_dat_o`	32	Output	Read data to the bus master (registered).
`wb_sel_i`	4	Input	Byte select. Present for compatibility; full-word writes are assumed.
`wb_we_i`	1	Input	Write enable: 1 = write cycle, 0 = read cycle.
`wb_stb_i`	1	Input	Strobe: selects the core for this transfer.
`wb_cyc_i`	1	Input	Valid bus cycle in progress.
`wb_ack_o`	1	Output	Acknowledge: a one-cycle pulse completing each access.
`wb_err_o`	1	Output	Bus error. **Tied to 0** in this implementation.
`wb_int_o`	1	Output	Interrupt: set at the end of a transfer when CTRL.IE = 1, cleared by the next bus access.

Table 1. Wishbone interface signals.

3.2 Register map

Address	Register	Access	Description
`0`	TX / RX	W / R	Write: byte to transmit (bits 7:0). Read: last byte received (bits 7:0).
4	CTRL	R/W	Bit 8 GO (start transfer, cleared by hardware when done); bit 9 RX_NEGEDGE; bit 10 TX_NEGEDGE; bit 12 IE (interrupt enable).
5	DIVIDER	R/W	SCLK divider (16 bits): $f_{SCLK} = f_{wb_clk} / (2 \times (DIVIDER + 1))$.
6	SS	R/W	Slave-select mask. `ss_pad_o` is the inverse (active low). Released automatically after each transfer.

Table 2. Register map.

3.3 Internal architecture

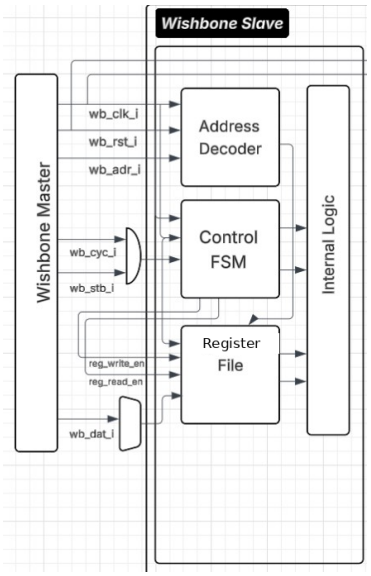


Figure 2. Wishbone slave internal architecture.

The address decoder selects a register from `wb_adr_i` when `wb_cyc_i` and `wb_stb_i` are both high. Writes update the register file on the same clock edge. Reads are registered, so `wb_dat_o` is valid one cycle after the access. In the RTL, the "control" logic is a registered single-cycle acknowledge (`ack <= cyc & stb & ~ack`) rather than a multi-state FSM.

4. Clock Generator (spi_clgen.v)

The clock generator produces SCLK by dividing `wb_clk_i`. A down-counter is reloaded with the `divider` value; each time it reaches zero, SCLK toggles. One SCLK half-period therefore lasts (DIVIDER + 1) system clock cycles. Changing DIVIDER lets the same master talk to fast and slow slaves.

Alongside SCLK, the block produces `pos_edge` and `neg_edge`, one-cycle pulses that mark each SCLK rising and falling edge. The serial interface uses these pulses to decide when to shift and when to sample. When no transfer is in progress, SCLK is held low, so the bus idles in **SPI mode 0** (CPOL = 0).

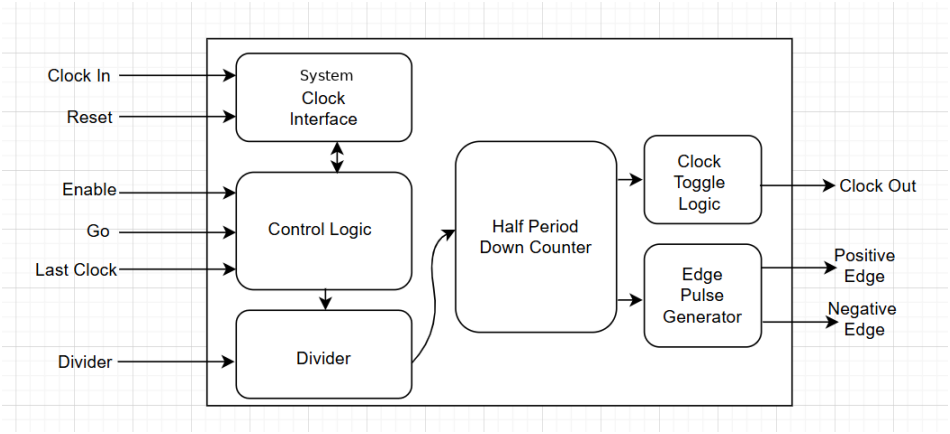


Figure 3. Clock generator internal architecture.

Signal	Direction	Description
`clk_in`	Input	System clock (`wb_clk_i`).
`rst`	Input	Asynchronous reset, active high.
`enable`	Input	Run the clock: connected to `tip`. When low, the counter reloads and SCLK is held low.
`go`	Input	Start transfer. Connected but not used by this implementation.
`last_clk`	Input	Last bit of the transfer. Connected but not used by this implementation.
`divider[15:0]`	**Input**	Clock divider value from the DIVIDER register.
`clk_out`	Output	Divided serial clock, driven to `sclk_pad_o`.
`pos_edge`	Output	One-cycle pulse marking an SCLK rising edge.
`neg_edge`	Output	One-cycle pulse marking an SCLK falling edge.

Table 3. Clock generator signals.

5. Serial Interface / Shift Register (spi_shift.v)

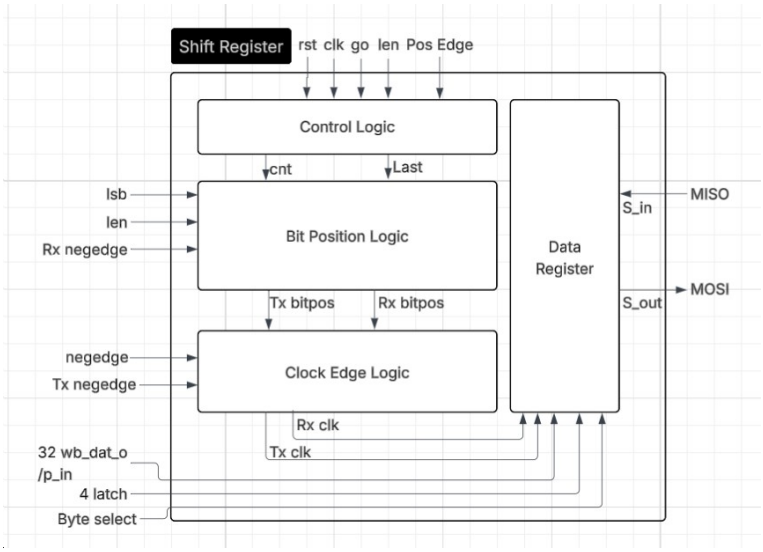


Figure 4. Shift register internal architecture.

In SPI, both master and slave hold a shift register. On each SCLK cycle, the master shifts one bit out on MOSI while the slave shifts one bit out on MISO, and each side shifts in the bit from the other. After eight clock cycles of an 8-bit transfer, the two registers have exchanged contents: this is full-duplex operation.

In this implementation, writing the TX register loads `tx_shift` (only while idle). Setting CTRL.GO starts a transfer: `tip` goes high and a bit counter is loaded with 8. On each shift edge the MSB of `tx_shift` is driven onto MOSI; on each sample edge the MISO bit is shifted into `rx_shift`. After the eighth sample, the received byte is copied to `p_out` and `tip` falls. `tx_negedge` and `rx_negedge` select whether shifting and sampling happen on the rising or falling SCLK edge. MISO passes through one register stage (`miso_d`) before sampling.

Signal	Direction	Description
clk	Input	System clock.
rst	Input	Asynchronous reset, active high.
latch[3:0]	Input	Load the TX shift register from p_in (bit 0 used).
byte_sel[3:0]	Input	Byte-lane select. Not used (8-bit design).
len	Input	Character length. Not used: length is fixed at 8 bits.
lsb	Input	LSB-first select. Not used: data is always MSB first.
go	Input	Start transfer (CTRL.GO).
pos_edge	Input	SCLK rising-edge pulse from the clock generator.
neg_edge	Input	SCLK falling-edge pulse from the clock generator.
rx_negedge	Input	1 = sample MISO on the falling SCLK edge; 0 = rising edge.
tx_negedge	Input	1 = drive MOSI on the falling SCLK edge; 0 = rising edge.
tip	Output	Transfer in progress.
last	Output	Last bit of the transfer.
p_in[31:0]	Input	Parallel TX data (bits 7:0 used).
p_out[127:0]	Output	Parallel RX data (bits 7:0 used; upper bits are 0 and removed by synthesis).
sclk	Input	Serial clock. Connected but not used (edge pulses are used instead).
miso	Input	Serial data in.
mosi	Output	Serial data out.

Table 4. Shift register signals.

6. Complete SPI–Wishbone Architecture

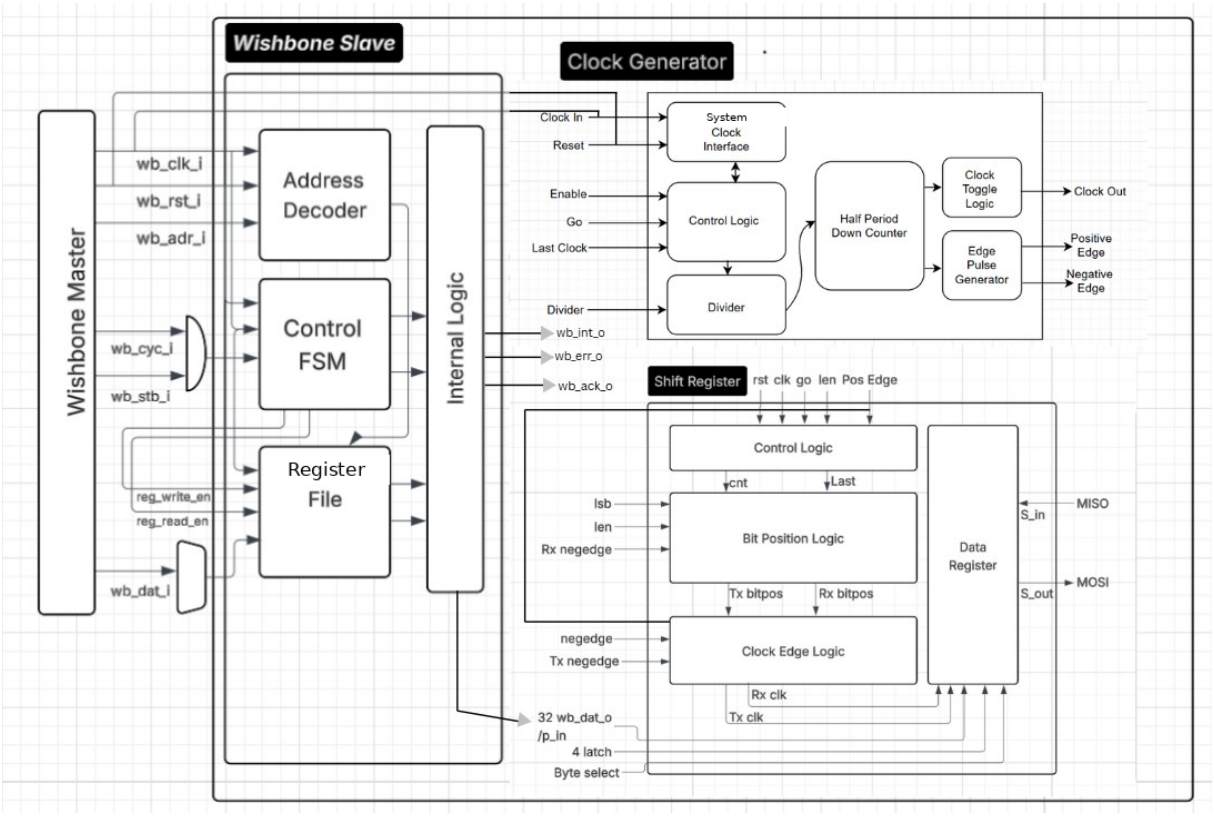


Figure 5. Complete SPI–Wishbone internal architecture.

The complete core acts as a bridge: the processor writes a byte and control settings over the parallel Wishbone bus, the shift register converts it to a serial stream on MOSI, and the byte received on MISO is converted back to parallel form and read over the bus. An interrupt can signal completion so the processor does not have to poll.

7. SPI Timing

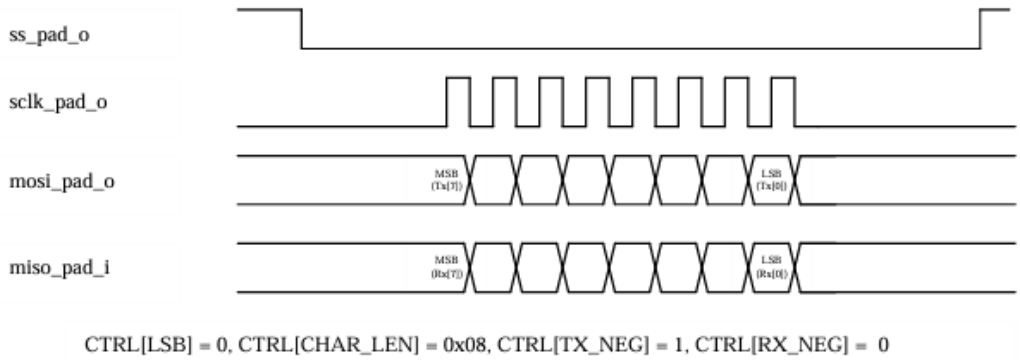


Figure 6. Reference SPI timing for an 8-bit, MSB-first transfer (adapted from the OpenCores SPI specification).

Slave select goes low before the first SCLK edge and returns high after the last. Data is transferred MSB first. The original OpenCores core also supports LSB-first and variable-length transfers; this implementation supports only 8-bit MSB-first transfers, so those modes are not shown.

8. Verification

The design is verified with a self-checking testbench (`tb_spi_top.v`) containing a Wishbone master bus model, which issues register reads and writes, and an SPI slave model, which returns a known byte on each transaction. The system clock is 100 MHz and DIVIDER = 2, giving SCLK = 100 / (2 × 3) ≈ 16.7 MHz.

Test	Sequence	Expected	Result
Transfer 1	Write DIVIDER = 2, TX = 0xA5, SS = 0x01, CTRL = 0x108 (GO, 8 bits); poll GO until clear; read RX.	MOSI shifts out 0xA5; slave returns 0xA5; RX = 0xA5	**PASS**
Transfer 2	Write TX = 0x00, SS = 0x01, CTRL = 0x108; poll GO; read RX.	Slave returns 0x3C; RX = 0x3C	**PASS**

Table 5. Directed tests and results (fixed RTL).

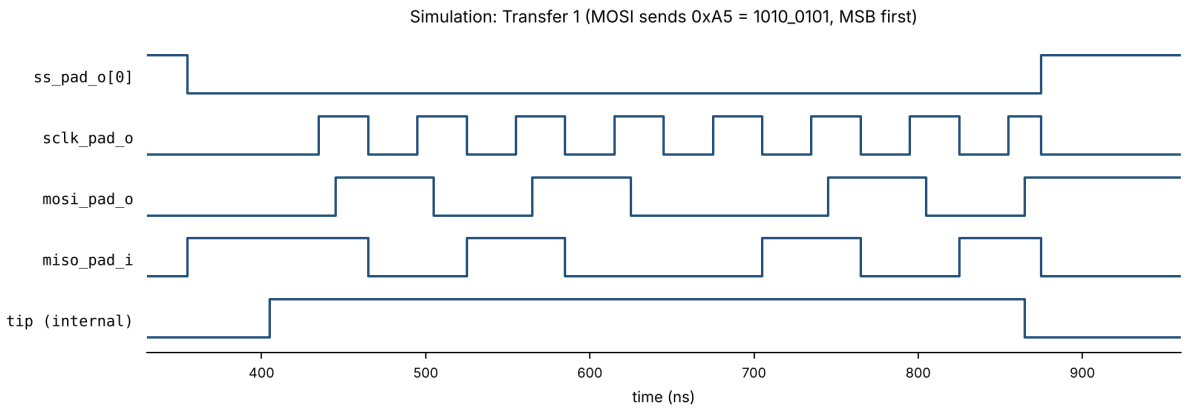


Figure 7. Simulated Transfer 1. MOSI carries 1-0-1-0-0-1-0-1 (0xA5), MSB first.

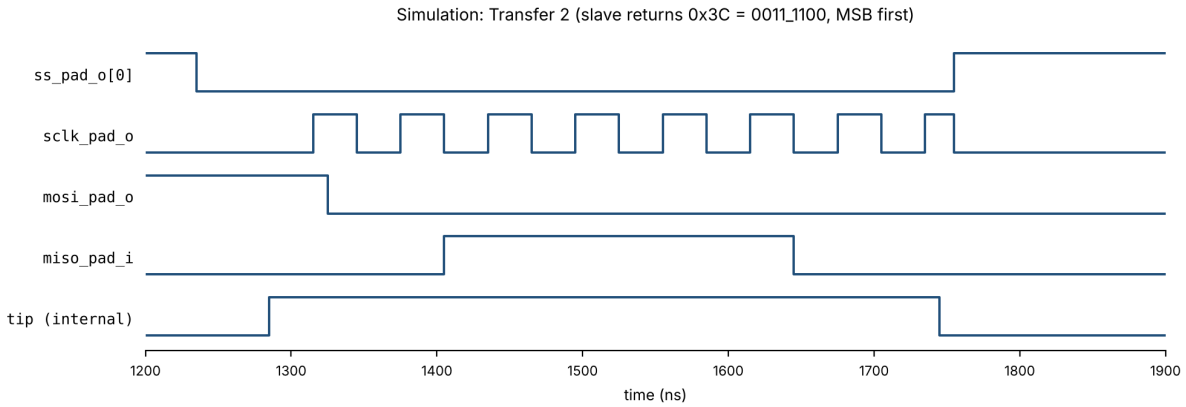


Figure 8. Simulated Transfer 2. MISO carries 0-0-1-1-1-1-0-0 (0x3C), sampled on each SCLK rising edge.

9. Bugs Found and Fixed

The design as first written passed Transfer 1 but **failed Transfer 2**, reading 0x01 instead of 0x3C, and could not be synthesized cleanly. Debugging in simulation found five problems:

#	Problem	Root cause and effect	Fix
1	Phantom transfer	CTRL.GO is cleared one clock after `tip` falls. In that cycle `go && !tip` was true, so an unwanted second 8-bit transfer started after every real one.	Added a `go_r` flag that marks a GO as already serviced; a new transfer needs a fresh GO.
2	SCLK idle level	An 8-bit transfer is 15 SCLK toggles, so SCLK was left high. The next transfer began on a falling edge and every bit shifted by one position (0x3C read as 0x78).	Hold SCLK low whenever the clock generator is disabled (SPI mode 0).
3	Multi-driven register	`tx_shift` was assigned in two `always` blocks. Simulation tolerates this, but synthesis reports multiple drivers.	Merged load and shift into one `always` block.
4	Wishbone ACK	ACK stayed high for consecutive cycles while `stb` was held.	Single-cycle pulse: `ack <= cyc & stb & ~ack`.
5	Wrong constraint format	Only a Vivado `.xdc` file existed. Cadence Genus, Innovus and Tempus read SDC, so the ASIC flow had no usable timing constraints.	Wrote `spi_top.sdc`: clock, IO delays, uncertainty and design-rule limits.

Table 6. Bugs found and fixed. Line-by-line changes are in rtl/CHANGES_*.diff in the repository.

10. Known Limitations and Future Work

- Transfers are fixed at 8 bits, MSB first. The `len`, `lsb` and `byte_sel` inputs are wired but unused, and `wb_err_o` is tied to 0.
- The testbench uses TX_NEGEDGE = 0, so MOSI changes on the SCLK rising edge. For strict SPI mode 0 slaves, set CTRL.TX_NEGEDGE = 1 so MOSI changes on the falling edge.
- The last SCLK high phase of a transfer is one system clock shorter than the others (20 ns instead of 30 ns in Figures 7 and 8), because SCLK is forced low as soon as the transfer ends. At high SCLK rates this should be checked against the slave's minimum high time.
- Next steps: synthesis in Cadence Genus, equivalence checking in Conformal, place and route in Innovus, signoff timing in Tempus, IR-drop analysis in Voltus and DRC/LVS in Pegasus on a 45 nm PDK. Scripts are in the repository's `scripts/` folder.

References

- S. Srot, *SPI Master Core Specification*, OpenCores, opencores.org/projects/spi.
- *Wishbone System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores*, OpenCores.